COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	August 24, 2026	**DATE SUBMITTED:**	August 27, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Hands-On Activity 6.1 Searching Techniques**

1. PROCEDURE OUTPUT

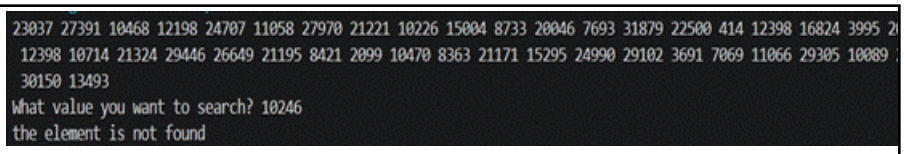
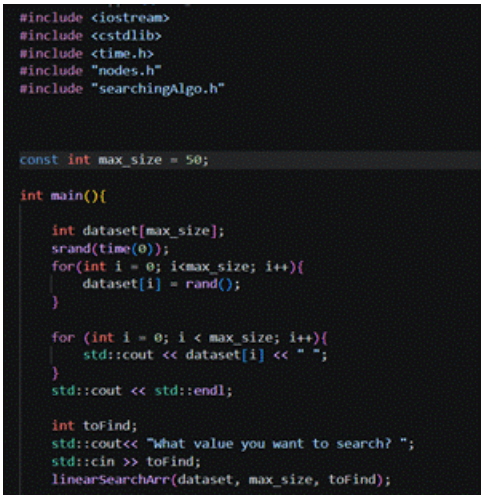
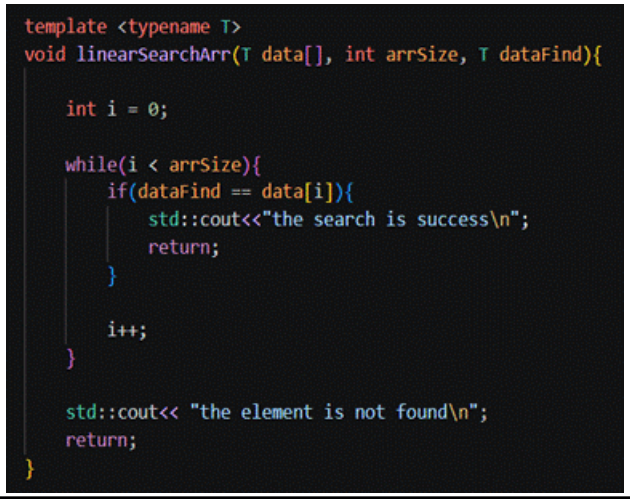
Screenshot	  
Observations	In the code, I've used the **library cstdlib for the values in the array, which was generated randomly by the syntax for(int i=0; i<max_size; i++){ dataset[i] = rand(); }. Also, time() returns the current system time in seconds by the use of the library <time.h>, and it provides a value that changes continuously every passing second**. **Passing the dynamic value to srand()**, which **implements the pseudo-random number generator, ensures that rand() produces a unique sequence of numbers every time the program executes**. In **linearSearchArr, which its parameters are T data[], int arrSize, and T dataFind, which is needed for our while loop. The while loop consists of loops through the array by i<arrSize and checking if dataFind == data[i], which prints "the search is successful\n" and immediately exits the function via return**. If **the loop continues without finding a match, the execution moves past the while loop and prints "the element is not found \n", and returns**.

Table 6-1. Data Generated and Observations.

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"


const int max_size = 50;

int main(){

    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i<max_size; i++){
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int toFind;
    std::cout<< "What value you want to search? ";
    std::cin >> toFind;
    linearSearchArr(dataset, max_size, toFind);
```

```
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){

    int i = 0;

    while(i < arrSize){
        if(dataFind == data[i]){
            std::cout<<"the search is success\n";
            return;
        }

        i++;
    }

    std::cout<< "the element is not found\n";
    return;
}
```

Output

```
23037 27391 10468 12198 24707 11058 27970 21221 10226 15004 8733 20046 7693 31879 22500 414 12398 1
 12398 10714 21324 29446 26649 21195 8421 2099 10470 8363 21171 15295 24990 29102 3691 7069 11066 2
 30150 13493
What value you want to search? 10246
the element is not found
```

Observations	In the main code, I've **used the library cstdlib for the values in the array, which was generated randomly by the syntax for(int i=0; i<max_size; i++){ dataset[i] = rand(); }. Also, time() returns the current system time in seconds by the use of the library <time.h>, and it provides a value that changes continuously every passing second. Passing the dynamic value to srand(), which implements the pseudo-random number generator, ensures that rand() produces a unique sequence of numbers every time the program executes, which displays on the output that there are random numbers. In the int toFind function, I've used the user input implementation (std::cin >> toFind).** In the **linearSearchArr function**, which **its parameters are T data[], int arrSize, and T dataFind, which is needed for our while loop. The while loop consists of loops through the array by i<arrSize and checking if dataFind == data[i], which prints "the search is successful\n" and immediately exits the function via return. If the loop continues without finding a match, the execution moves past the while loop and prints "the element is not found \n", and returns, which shows in the output when I type the number 10246, and there is no number like that in the randomly generated array, so the result is "the element is not found"**.

Table 6-2a. Linear Search for Arrays

Code

```
//Linked List
Node<char>* name1 = new_node('J');
Node<char>* name2  = new_node('I');
Node<char>* name3  = new_node('M');
Node<char>* name4  = new_node('L');
Node<char>* name5  = new_node('O');
Node<char>* name6  = new_node('R');
Node<char>* name7  = new_node('D');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

std::cout<<"Linear Search for Linked List Result: ";
linearSearchLL(name1, 'D');
std::cout<<"Linear Search for Linked List Result: ";
linearSearchLL(name1, 'x');
```

```
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
public:
    T data;
    Node *next;
};

template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode -> data = newData;
    newNode -> next = NULL;
    return newNode;
}

#endif
```

```
template <typename T>
void linearSearchLL (Node<T> *node, T dataFind){
    //Repeat the loop until the current node is not null
    while (node != nullptr){

    if(dataFind == node -> data){
        std::cout<<"the search is a success\n";
        return;
    }
    node = node -> next;
    }
    std::cout<<"The element is not found\n";
    return;
}
```

Output	Linear Search for Linked List Result: the search is a succe Linear Search for Linked List Result: The element is not fo
Observations	In the code, I've **correctly built a singly linked list of character nodes (' J ' , ' I ' , ' M ' , ' L ' , ' O ' , ' R ' , ' D ') and correctly execute the sequential search on it . When I'm running linearSearchLL(name1, 'D'), the program goes through the nodes from the head (name1) until it finds the matching character at the last node (name7), and correctly outputs "Linear Search for Linked List Result: the search is a success"**. However, **if we are looking for 'x', the function goes through the entire list and does not find a match, and reaches the terminating nullptr and correctly outputs "Linear Search for Linked List Result: The element is not found". This shows that the templated node structure is allocating memory correctly and dynamically and that the linear search is working for both successful data finding and error-case handling.**

Table 6-2b. Linear Search for Linked List

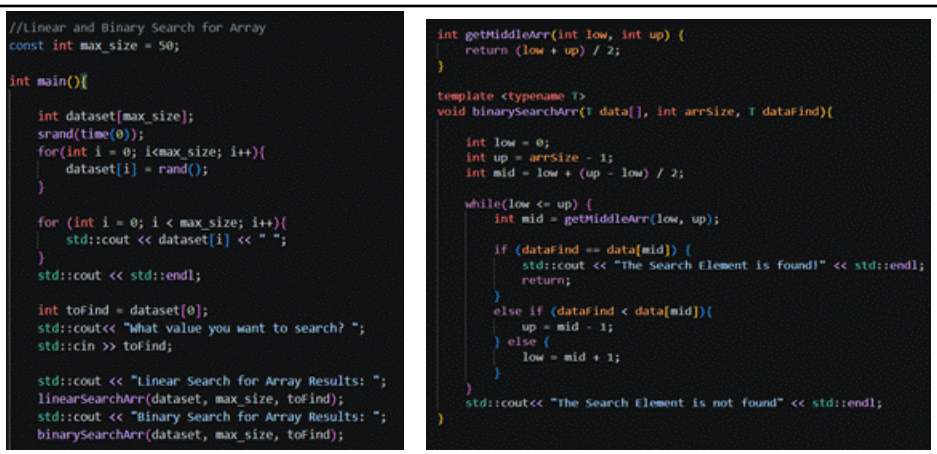
Code	
Output	21454 28033 14594 476 1446 32241 28169 9994 25496 11499 29424 7701 15229 878 5389 1544 23297 22538 20993 8724 1122 149 4761 145 488 3314 32298 20969 22825 4577 14974 19919 7187 18833 9576 7514 10707 9070 9659 11492 6891 32066 27685 19506 6812 17822 7173 1 What value you want to search? 10246 Linear Search for Array Results: the element is not found Binary Search for Array Results: The Search Element is not found
Observations	I think the program uses an **array of 50 numbers**, randomly generated by the **rand ()** function and implemented with system time. Then it prints the numbers contained in the array. Additionally, it asks a user to enter a number to search (in this case 10246) using the **binary search algorithm**. The **binary search algorithm** incorrectly identifies that the number is not contained in the array. I think the binary search algorithm incorrectly fails to find the search element (10246) because **it usually requires an ordered input array**, which is not provided in my code. Therefore, to make sure that the binary search algorithm works properly, we **should sort the input array before passing it to the binarySearchArr function**.

Table 6-3a. Binary Search for Arrays

Code

```
//Linear and Binary Search for Linked List
Node<char>* name1 = new_node('J');
Node<char>* name2 = new_node('I');
Node<char>* name3 = new_node('M');
Node<char>* name4 = new_node('L');
Node<char>* name5 = new_node('O');
Node<char>* name6 = new_node('R');
Node<char>* name7 = new_node('D');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;
```

```
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
public:
    T data;
    Node *next;
};

template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode -> data = newData;
    newNode -> next = NULL;
    return newNode;
}

#endif
```

```
std::cout<<"Binary Search for Linked List Result: ";
binarySearchLL(name1, 'J');
std::cout<<"Binary Search for Linked List Result: ";
binarySearchLL(name1, 'C');
```

```
template <typename T>
Node<T>* getMiddleLL(Node<T>* start, Node<T>* last){
    if (start == nullptr) return nullptr;

    Node<T>* slow = start;
    Node<T>* fast = start;


    while(fast != last && fast -> next != last){
        //move slow (one step)
        slow = slow -> next;
        //move slow (two steps)
        fast = fast -> next -> next;
    }
    //when fast reaches the end
    // slow points to the middle node
    return slow;
}
```

```
template <typename T>
void binarySearchLL (Node<T> *start, T findData){
    Node<T>* low = start;
    Node<T>* up = nullptr;

    while (low != up && (low == nullptr || low -> next != up || low != up)){
        if (low == nullptr) break;

        Node<T>* mid = getMiddleLL(low, up);

        if (mid == nullptr) break;


        if (findData == mid -> data){
            std::cout << "Search element is found!" << std::endl;
            return;
        }
        else if (findData < mid -> data) {
            up = mid;
        } else {
            low = mid -> next;
        }
    }
    std::cout << "The Search Element is not found" << std::endl;
}
```

Output

```
Binary Search for Linked List Result: The Search Element is not found
Binary Search for Linked List Result: The Search Element is not found
```

Observations

In my opinion, the code is wrong due to its attempt to perform a binary search **(getMiddleLL)** of the **singly linked list (SLL).** The analysis of the code revealed that there is one issue that can lead to the obtained result. This is known that the code is for the binary search, it is necessary to have **an ordered list of the elements** (for example, “D”, “I”, “J”, “L”, “M”, “O”, “R”), **while the SLL in the program is initialized with the unordered data (“J”, “I”, “M”, “L”, “O”, “R”, “D”)**. As a result, the **comparisons (if (findData < mid->data) { })** always return **false**, and the **binary search is not performed correctly even though “J” is the first element**. Consequently, the program displays **“The Search Element is not found”**. For this reason, it is more efficient to use the **linear search instead of the binary one for the linked list**.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

7. Supplementary Activity

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

```
const int size = 10;

int main() {
    // Problem 1 & 2 Dataset
    int dataset[size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    Node<int>* head = new Node<int>(dataset[0]);
    Node<int>* tail = head;
    for (int i = 1; i < size; i++) {
        tail->next = new Node<int>(dataset[i]);
        tail = tail->next;
    }


    std::cout << "\n----------PROBLEM 1 & 2----------\n\n";

    std::cout << "Linear Search for '18' in Array:\n";
    linearSearchArr(dataset, size, 18);

    std::cout << "Linear Search for '18' in Linked List:\n";
    linearSearchLL(head, 18);
```

```
//Helper Functions
int getMiddleArr(int low, int high) {
    return low + (high - low) / 2;
}
```

```cpp
template <typename T>
Node<T>* getMiddleLL(Node<T>* start, Node<T>* last) {
    Node<T>* slow = start;
    Node<T>* fast = start;
    while (fast != last && fast->next != last) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

//Linear Search Functions
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind) {
    int counter = 0;
    for (int i = 0; i < arrSize; i++) {
        counter++;
        if (data[i] == dataFind) {
            std::cout << "Search Success\n";
            std::cout << "Comparison Count: " << counter << "\n";
            return;
        }
    }
    std::cout << "Element not found\nComparison Count: " << counter << "\n";
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T>* node, T dataFind) {
    int counter = 1;
    while (node != nullptr) {
        if (node->data == dataFind) {
            std::cout << "Search Success\n";
            std::cout << "Comparison Count: " << counter << "\n";
            return;
        }
        node = node->next;
        counter++;
    }
    std::cout << "Element not found\nComparison Count: " << counter - 1 << "\n";
}
```

Supplementary > C nodesSupp.h > ...

```cpp
#ifndef NODESSUPP_H
#define NODESSUPP_H

template <typename T>
struct Node {
    T data;
    Node<T>* next;

    Node(T val) : data(val), next(nullptr) {}
};

#endif
```

CODE

```
Linear Search for '18' in Array:
Search Success
Comparison Count: 2
Linear Search for '18' in Linked List:
Search Success
Comparison Count: 2

Modified Linear Search for counting '18's in Array:
Count: 2
Modified Linear Search for counting '18's in Linked List:
Count: 2
```

OUTPUT

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

```cpp
//Modified Linear Search (Count Occurrences)
template <typename T>
void linearSearchArrMOD(T data[], int arrSize, T dataFind) {
    int i = 0;
    int counter = 0;
    while (i < arrSize) {
        if (data[i] == dataFind) {
            counter++;
        }
        i++;
    }
    std::cout << "Count: " << counter << "\n";
}
```

```cpp
template <typename T>
void linearSearchLLMOD(Node<T>* node, T dataFind) {
    int counter = 0;
    while (node != nullptr) {
        if (node->data == dataFind) {
            counter++;
        }
        node = node->next;
    }
    std::cout << "Count: " << counter << "\n";
}
```

```cpp
std::cout << "\nModified Linear Search for counting '18's in Array:\n";
linearSearchArrMOD(dataset, size, 18);

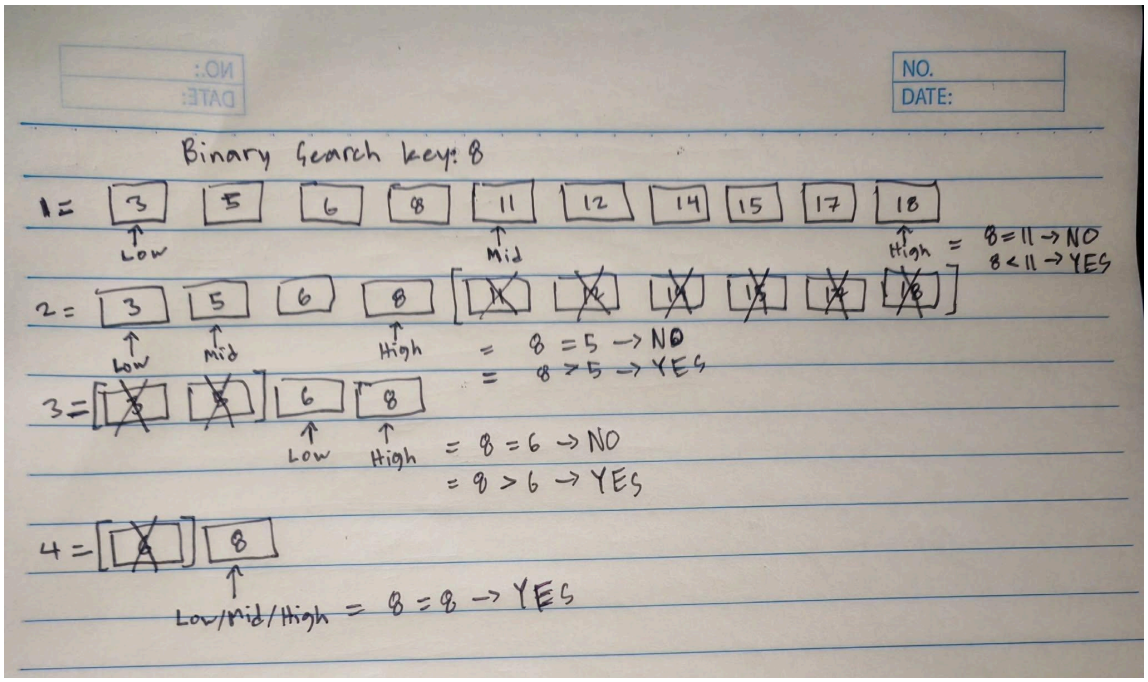
std::cout << "Modified Linear Search for counting '18's in Linked List:\n";
linearSearchLLMOD(head, 18);
```

CODE

```
Modified Linear Search for counting '18's in Array:
Count: 2
Modified Linear Search for counting '18's in Linked List:
Count: 2
```

OUTPUT

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.



DIAGRAM

```cpp
// Problem 3 & 4 Dataset
int dataset2[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
Node<int>* head2 = new Node<int>(dataset2[0]);
Node<int>* tail2 = head2;
for (int i = 1; i < size; i++) {
    tail2->next = new Node<int>(dataset2[i]);
    tail2 = tail2->next;
}

std::cout << "\n----------PROBLEM 3 & 4----------\n\n";
std::cout << "Binary Search for '8' in Array:\n";
binarySearchArr(dataset2, size, 8);

std::cout << "Binary Search for '8' in Linked List:\n";
binarySearchLL(head2, 8);
```

```cpp
//Iterative Binary Search
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
    int low = 0;
    int high = arrSize - 1;
    while (low <= high) {
        int middle = getMiddleArr(low, high);
        if (data[middle] == dataFind) {
            std::cout << "Search Success\n";
            return;
        } else if (data[middle] > dataFind) {
            high = middle - 1;
        } else {
            low = middle + 1;
        }
    }
    std::cout << "Search Failed\n";
}

template <typename T>
void binarySearchLL(Node<T>* head, T dataFind) {
    Node<T>* start = head;
    Node<T>* last = nullptr;
    while (start != last) {
        Node<T>* middle = getMiddleLL(start, last);
        if (middle == nullptr) break;

        if (middle->data == dataFind) {
            std::cout << "Search Success\n";
            return;
        } else if (middle->data > dataFind) {
            last = middle;
        } else {
            start = middle->next;
        }
    }
    std::cout << "Search Failed\n";
}
```

CODE

```
----------PROBLEM 3 & 4----------

Binary Search for '8' in Array:
Search Success
Binary Search for '8' in Linked List:
Search Success
```

OUTPUT

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

```
std::cout << "\nModified Binary Search for counting '8's in Array:\n";
binarySearchArrMOD(dataset2, size, 8, 0, size - 1);
```

```cpp
//Recursive Binary Search
template <typename T>
void binarySearchArrMOD(T data[], int arrSize, T dataFind, int low, int high) {
    if (low <= high) {
        int middle = getMiddleArr(low, high);
        if (data[middle] == dataFind) {
            std::cout << "Search Success\n";
            return;
        } else if (data[middle] > dataFind) {
            high = middle - 1;
            binarySearchArrMOD(data, arrSize, dataFind, low, high);
            return;
        } else {
            low = middle + 1;
            binarySearchArrMOD(data, arrSize, dataFind, low, high);
            return;
        }
    }
    std::cout << "Search Failed\n";
}

#endif
```

CODE

```
Modified Binary Search for counting '8's in Array:
Search Success
```

OUTPUT

CONCLUSION/LEARNINGS/ANALYSIS	Based on my Analysis, **implementing the two searching algorithms for array and linked list really help me understand how the fundamental memory structures influence the development of searching algorithms, pointer manipulation, and even the control flow within the implementation**. Practicing the **Linear search (linearSearchLLMOD)** helped me **recognize the simplicity of such algorithms**, **as iterating through the array or linked list is linear**, which enabling me to **easily count the number**

	of duplicates. In contrast, implementing **Binary search (binarySearchArr and binarySearchLL)** demonstrated to me the **key differences in approaches for arrays and linked lists**. For instance, **arrays allow me to directly access the middle of the structure**, while **linked lists require me to develop helper functions like (getMiddleLL)** to **achieve similar functionality**. Moreover, the transition **from iterative approaches (binarySearchArr) to recursive ones (binarySearchArrMOD)** showed me **how recursion could be employed to implement the same binary search mechanism**. Most importantly, I saw **that using recursion influences the way I pass parameters (low, high) to the function as well as the control flow within the function**. Overall, I saw the **strong connection between the implementation of the algorithms and the data structures**. For example, **arrays are more flexible in terms of mathematical operations, while linked lists are more efficient when it comes to pointer manipulation**. In Conclusion, I've **got some knowledge when it comes to the Linear and Binary Search within Arrays and Linked List when it comes to their implementation of the functions and how to manipulate its pointers but I still have to improve my Pointer Manipulation technique**.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.